PROJECT PROPOSAL

Vashasetu

Turn any regional dialect into Standard Bangla — by voice, instantly.

Software Engineering
CSE 355

Student 1 (Team Member):	Sadia Akter Liza (ID: 0432320005101110)
Student 2 (Team Member):	Shobnom Sultana Muskan (ID: 0432320005101142)
Course Instructor:	Dr. Mahfida Amjad Dipa
Date of Submission:	September 1, 2026

i. Component Diagram

The component diagram shows Vashasetu's major software components and how they interact: the Web Frontend handles voice and text input/output, delegating speech recognition to the STT Service and speech synthesis to the TTS Service. The Matching Engine sits between the frontend and the Dictionary Database, resolving dialect text against stored word pairs using fuzzy matching. The Admin Panel provides a separate path for reviewing and approving community-contributed words before they reach the live dictionary.

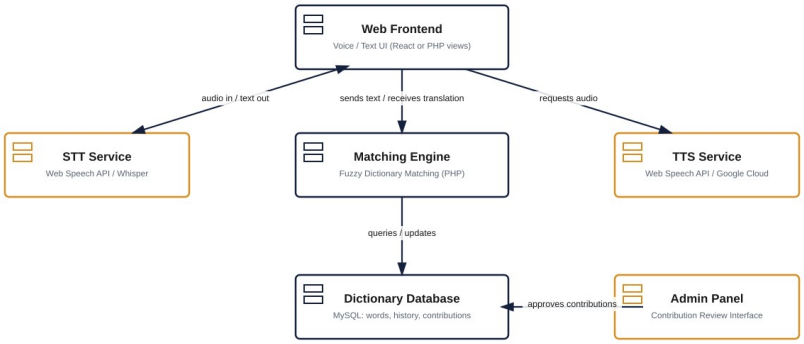


Figure 1: Vashasetu component diagram.

ii. Activity Diagram

The activity diagram traces a single user session from input to output. A user may either speak (routed through the STT engine) or type directly; both paths converge at the fuzzy-matching step. If a match is found, the system generates Standard Bangla text and speaks it back via TTS. If no match is found, the query is logged and the user is invited to contribute the missing word, keeping the dictionary self-improving over time.

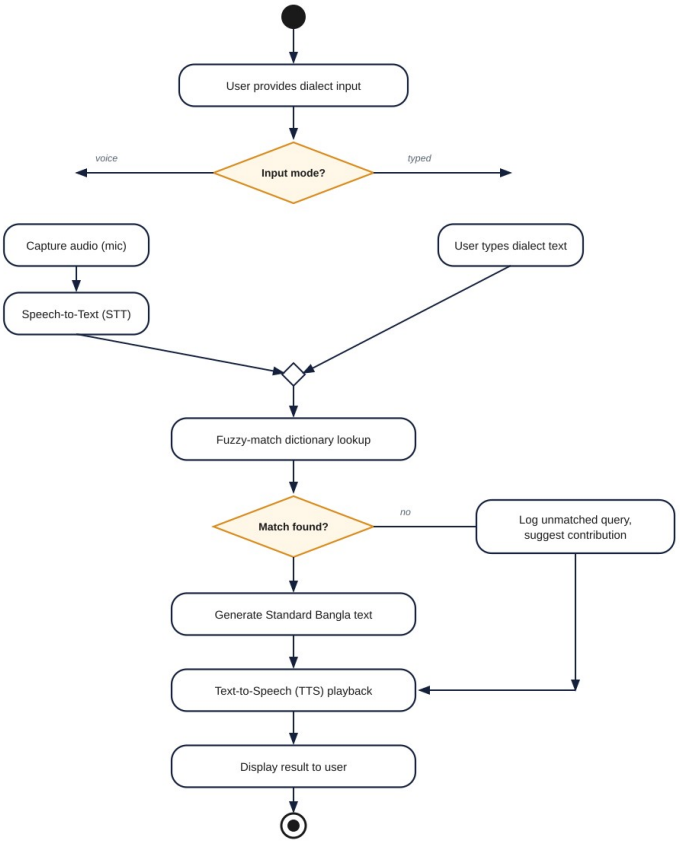


Figure 2: Vashasetu activity diagram — dialect-to-Standard-Bangla translation flow.

iii. Sequence Diagram

The sequence diagram details the runtime interaction between the User, the Web Frontend, and the three backend services (STT, Matching Engine, TTS) for a single translation request, along with the Matching Engine's query to the Dictionary Database. Dashed lines represent return messages; solid lines represent request calls.

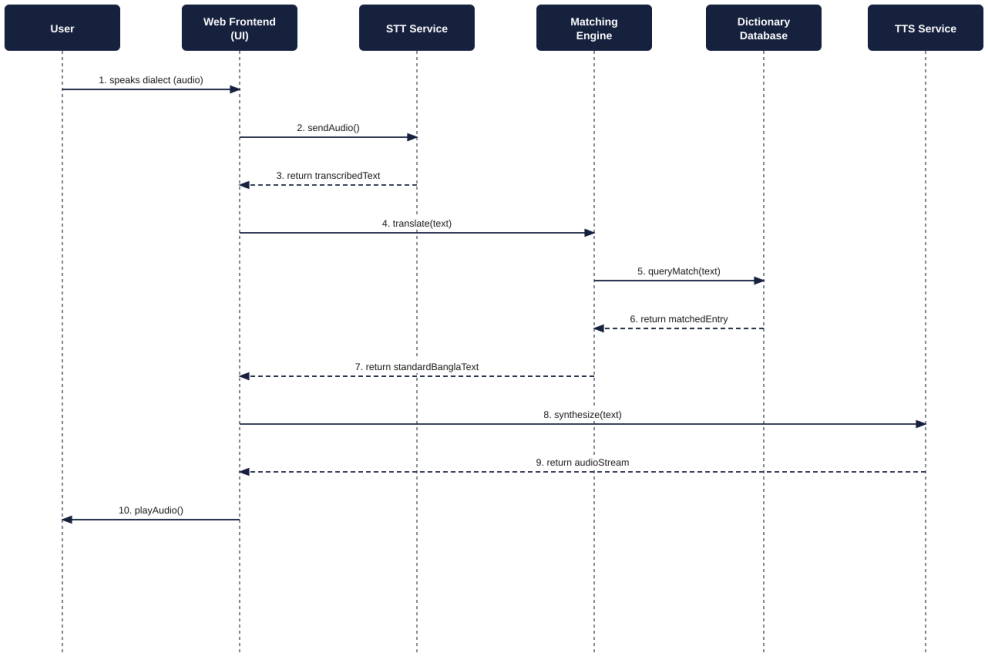


Figure 3: Vashasetu sequence diagram — end-to-end translation request.

iv. Fishbone (Ishikawa) Diagram

The fishbone diagram traces the root causes behind the central problem Vashasetu addresses — regional dialect speakers being excluded from digital services — across four categories: Technology (no dialect-aware tools exist), People (literacy and familiarity barriers), Process (no structured way to grow a dialect dictionary), and Society/Environment (structural factors like unstandardized dialects and rural connectivity).

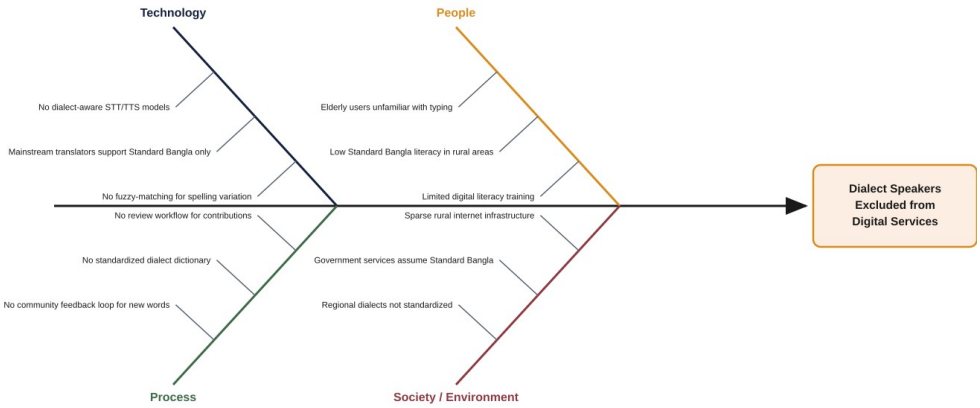


Figure 4: Fishbone diagram — root causes of dialect-speaker digital exclusion.

Tools & Development Environment

The table below summarizes the tools, technologies, and environment used to design, build, test, and deploy Vashasetu.

Category	Tool / Technology	Purpose
Frontend Development	HTML5, CSS3, JavaScript, PHP (view layer)	Voice/text UI: Translate, History, Contribute screens
Backend Development	PHP 8.x	REST-style API endpoints, business logic, fuzzy-matching engine
Database	MySQL 8.0	Dictionary storage, user data, translation history, contributions
Speech-to-Text (STT)	Web Speech API, OpenAI Whisper	Converts spoken dialect audio into text
Text-to-Speech (TTS)	Web Speech API, Google Cloud TTS	Converts translated Standard Bangla text into audio
UI/UX Design & Prototyping	Figma	Wireframes, mockups, and design system
Version Control	Git, GitHub	Source code management and team collaboration
Hosting & Deployment	Hostinger (shared hosting)	Production hosting for the web application and database
Development IDE	Visual Studio Code	Primary code editor for frontend and backend development
Operating System	Windows 10/11 (development), Linux (Hostinger server)	Local development and production server environment
Testing	Manual testing, browser DevTools	Unit and user-acceptance testing across the translation pipeline